Prediction

(P1) Nếu một key có dot product lớn hơn key khác, key đó sẽ nhận được attention weight lớn nhất trong toàn bộ sequence. Hàm softmax sử dụng hàm số mũ e^x là hàm đồng biến nghiêm ngặt ($s_a > s_b \Leftrightarrow e^{s_a} > e^{s_b}$), và mẫu số chuẩn hoá $\sum_j e^{s_j}$ là dương và dùng chung cho 1 key → score lớn nhất → tử số lớn nhất → attention weight lớn nhất. conf: 100%.

(P2) Nếu tất cả scores giống nhau, các attention weights sẽ bằng nhau hoàn toàn và tạo thành phân phối đều. Do $s_1 = s_2 = s_3 = s_4 = 1 \Rightarrow e^{s_i} = e^1$ thì

$\Rightarrow \alpha_i = \frac{e^1}{4e^1} = \frac{1}{4} = 0,25$ conf: 100%.

(P3) Nếu một score lớn hơn rất nhiều, $s = [1, 1, 10]$, attention gần như tập trung toàn bộ vào phần tử thứ 3, 2 phần tử đầu nhận trọng số rất nhỏ. Do hàm mũ khuếch đại mạnh → tỉ số lớn → softmax hoạt động tiệm cận với hàm argmax. conf: 100%.

(P4) Nếu sequence tăng từ 10 tokens → 1000 tokens, chi phí attention (chi phí tính toán + bộ nhớ lưu trữ ma trận attention) sẽ tăng gấp 10000 lần. Độ phức tạp của attention matrix giữa các token trong một chuỗi là $O(T^2)$. $T = 10 \rightarrow T = 1000$, số phần tử ~~tăng~~ và số phép tính tăng từ $10^2 = 100 \rightarrow 1000^2 = 1\,000\,000$. conf: 100%.

(P5) Attention không nhất thiết luôn tập trung vào đúng token mà con người đánh giá là quan trọng về mặt ngữ nghĩa hoặc ngữ pháp. - Attention weights được học và điều chỉnh qua gradient descent nhằm cực tiểu hoá hàm mất mát của bài toán cụ thể chứ không dựa trên quy tắc ngữ pháp.

- Phân phối attention phản ánh mối tương quan thống kê mà mô hình khai thác để tối ưu dự đoán, trong thực tế attention có thể bị phân tán vào dấu câu, stopwords hoặc tương quan ảo.

conf: 95%.

Được quét bằng CamScanner